Writing a Python script to create a bot controller using *python-library-telegram*

To create a bot controller script, we shall use a Python library *python-telegram-bot*; so download this using Pip:

```
pip install python-telegram-bot
```

This is shown in Figure 5.

To write the Python script, you can use any popular IDE. Some common ones are PyCharm, Visual Studio Code and Eclipse (with PyDev). You need to know some of the classes and methods of this library in order to truly understand how the code works. Remember, we are using the *polling* and not the *webhook* technique.

We shall use instances of (i.e., we will create objects of) two classes from the *telegram.ext* sub-module, namely, *telegram.ext.Updater* and *telegram.ext.Dispatcher*. The Updater class gets updates from Telegram and passes them on to the Dispatcher class.

When you create an Updater object, it will create a Dispatcher object for you and link them together with a Queue. This Dispatcher object can then be used to sort the updates fetched by the Updater according to the handlers you registered, and deliver them to a callback function that you defined.

The above para may be a bit confusing, so I have elaborated on it, as follows.

Step 1: Create an object (we will call it *my_updater*) of class *telegram.ext.Updater*. To do this, you need to import the Updater class from *telegram.ext*. Also, when you create your Updater object, namely *my_updater*, you will need the HTTP API key for the bot you created earlier. The code to create this object will be like what follows:

```
my_updater = Updater(token='your_bot_token', use_context=True)
```

> **Note:** In the example code on the website (Ref 2 & 3), this variable is often called Dispatcher, but I have used *my_dispatcher* to specify that this is not some method of a class but an object of class Updater, which I have created.

Step 2: Now, this Updater class has an attribute called Dispatcher. This attribute returns an object of class *telegram.ext.Dispatcher*. Don't get confused by this scheme. This is a typical method by which many libraries of Python work. So you can create an object of Dispatcher class (we will call it *my_dispatcher*) by using the *dispatcher* attribute of the object of the Updater class (which here, is *my_updater*). The following code snippet shows this:

```
my_dispatcher = my_updater.dispatcher
```

Step 3: Here, we will write a function named *start*. So, whenever the bot user types a message */start*, this function gets triggered. The way the Telegram package is written is such that when you write a function (like */start* here), the Telegram server will pass it two attributes. So you must have two attributes in your function definition and by convention, these are called *update* and *context*. I don't want to get too technical but for those interested, the attribute *update* will get an object of class *telegram.update.Update*. While the attribute *context* will get an object of class *telegram.ext.callbackcontext.CallbackContext*.

The attribute *update* is actually a Python dictionary. You can confirm this by putting a *print(update)* command somewhere in your *get()* function. This dictionary has a *key* named *effective_chat* and the *value* of this key is another dictionary, which contains *keys* like *ID*, *first_name*, *last_name*, etc. So, using a format of type *update.effective_chat.id*, you can get the ID of the chat. In our bot code, we will 'extract' three parameters of the bot user — (1) *id*, (2) *first_name*, and (3) *last_name*.

In this step, we will do the following things:

- Extract *id*, *first_name* and *last_name* from the parameter *update*.
- We will use the *first_name* and *last_name* to send a reply back to the bot user.
- To send a reply back to the bot user, we need to use the following command:

```
context.bot.send_message(chat_id, text=out_text)
```

To make the code more readable and easier to follow, I have split this in two lines as:

```
my_bot = context.bot
my_bot.send_message(chat_id, text=out_text)
```

Figure 5: Downloading the Python library *python-telegram-bot*